Université d'Ottawa
Faculté de génie

School of Electrical
Engineering and
Computer Science

uOttawa
L'Université canadienne
Canada's university

University of Ottawa
Faculty of Engineering

École de science
informatique et de
génie électrique

CSI2520 Paradigmes de programmation
CSI2120 Programming Paradigms

75 minutes
Prof: Robert Laganière

Exercices à trous:

- Un énoncé par ligne
- Vous pouvez laisser certaines lignes blanches
- Vous ne pouvez écrire en dehors des lignes

Fill-in-the-blank questions:

- One statement per line
- You can leave some lines blank
- You cannot add statement outside these blank lines

Question 1 [5 points]

Dans le programme Go ci-suivant, la fonction `subtractMean` est appelée en temps que goroutine avec trois slices différents. Ajouter les mécanismes de synchronisation nécessaires afin de s'assurer que ces goroutines soient complétées avant d'afficher les résultats.

…….

In the following Go program, the `subtractMean` is called as a goroutine on three different slices. Add the synchronization mechanisms required to make sure that these goroutines terminate before the results are displayed.

```
func subtractMean(data []float64,_____________________________________)
{

    ____________________________________________________

    ____________________________________________________

    mean := computeMean(data)
    for i := range data {
         data[i] -= mean
    }

    ____________________________________________________
}
```

```
func main() {
     numbers1 := []float64{10, 20, 30, 40, 50}
     numbers2 := []float64{14, 20, 10, 20, 30, 40, 50}
     numbers3 := []float64{34, 22, 40, 50}



     ____________________________________________________________



     ____________________________________________________________

     // Start goroutines
     go subtractMean(numbers1, ______________________________)

     go subtractMean(numbers2, ______________________________)

     go subtractMean(numbers3, ______________________________)

     // Wait for goroutine to complete



     ____________________________________________________________



     ____________________________________________________________

     // Print the modified slices
     fmt.Println("Slice after processing:", numbers1)
     fmt.Println("Slice after processing:", numbers2)
     fmt.Println("Slice after processing:", numbers3)
}
```

<u>Question 2</u> [7 points]

Le programme de la page suivante montre un fil générateur produisant les nombres de la suite de Fibonacci.

La sortie obtenue est comme ci-dessous.

…….

The program on next page includes a generator thread producing the number of the Fibonacci series. The output is as follows:

```
1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
End of FibGenerator...
End of main...
```

a) Créer un module filtre qui se connectera à ce générateur en tant que goroutine et qui éliminera les nombres se terminant par 3, 5 ou 7, i.e.

```
if mod := fib%10 ; mod==3 || mod==5 || mod ==7 {
```

Écrire votre réponse à la page 6.

……

Create a filter stage that connects to the generator as a goroutine and that will eliminate numbers terminating with 3, 5 or 7 (see statement above). Write your answer on page 6.

b) Modifier la fonction `main` afin de connecter votre module filtre au channel du fil générateur, puis lire et afficher les 15 premières valeurs reçues. La sortie sera alors comme ci-dessous. Écrire votre réponse à la page 7.

…….

Modify the `main` function in order to connect your filter stage to the output channel of the generator thread. Read the 15 first values and print them to the console. The output will be as follows. Write your answer on page 7.

```
1 2 8 21 34 89 144 610 2584 4181 10946 17711 46368 196418 317811
```

```
func fibbonacciGenerator(wg *sync.WaitGroup, stop <-chan bool) <-chan int64 {
      intStream := make(chan int64)
      go func() {
            defer wg.Done()
            defer close(intStream)
            defer fmt.Println("\nEnd of FibGenerator...")

            var i0, i1, fib int64= 0, 1, 0
            for {
               fib= i0+i1
               i0= i1
               i1= fib
               select {
                  case <-stop:
                     return
                  case intStream <- fib:
               }
            }
      }()
      return intStream
}

func main() {
      stop := make(chan bool)
      defer close(stop)
      var wg sync.WaitGroup
      wg.Add(1)
      intChannel := fibbonacciGenerator(&wg, stop)

      for i:=0; i<15; i++ {
        fmt.Printf("%d ", <-intChannel)
      }
      stop <- true      // stop the thread
      wg.Wait()   // synchronisation
      fmt.Println("\nEnd of main...")
}
```

```
func filter357(_____________________________________________________)
                                                        <-chan int64
{
     intStream := make(chan int64)
     go func() {

     }()
     return intStream
}
```

```
func main() {
     stop := make(chan bool)
     defer close(stop)
     var wg sync.WaitGroup



     ____________________________________________________________

     intChannel := fibbonacciGenerator(&wg, stop)



     ____________________________________________________________



     ____________________________________________________________



     ____________________________________________________________

      for i:=0; i<15; i++ {
       fmt.Printf("%d ",_____________________________________________)
     }



     ____________________________________________________________



     ____________________________________________________________
     wg.Wait()      // synchronisation
     fmt.Println("\nEnd of main...")
}
```

<u>Question 3</u> [4 points]

Vous trouverez à la page suivante, la réalisation de l'algorithme de tri fusion utilisant des fils concurrents. En effet, chaque appel récursif à `MergeSort` créé potentiellement deux nouvelles goroutines. Pour fonctionner correctement, la fonction `MergeSort` a besoin de deux instances de `WaitGroup`, l'une étant reçue et l'autre étant envoyée aux goroutines. La fonction `Merge`, non-montrée ici, effectue simplement la fusion de deux tableaux triés en un seul tableau trié.

a) Pour que le tri fonctionne correctement, il manque des appels aux méthodes `Wait()` et/ou `Done()` des `WaitGroup`. Ajouter les appels requis à ces méthodes sur l'une ou l'autre des lignes montrées.

b) Pour l'exemple du tableau montré dans la fonction `main`, combien de goroutines seront créées au total afin de trier ce tableau?

_____

……

The next page shows an implementation of the Merge Sort algorithm using concurrent threads. Each recursive call to `MergeSort` potentially creates two new concurrent goroutines. To work properly, this `MergeSort` function needs to create two instances of `WaitGroup`, one being received as parameter and the other one created and sent to the goroutines. The not-shown `Merge` function simply perform the merge of two sorted arrays into one sorted array.

a) For this sort to work as expected, calls to the `Wait()` and/or `Done()` methods of the `WaitGroup` are missing. Add the required calls to these methods on the correct blank lines.

b) For the example of the array shown in the main function, how many goroutines will be created in order to completely sort this array?

_____

```
func MergeSort(arr []int, wg *sync.WaitGroup) []int {

     ____________________________________________________________

     // Base case: array of  one or zero elements
     if len(arr) <= 1 {
          return arr
     }

     // Split the array into two halves
     mid := len(arr) / 2
     left, right := arr[:mid], arr[mid:]

     // Use WaitGroup to synchronize goroutines
     var wg2 sync.WaitGroup
     wg2.Add(2)

     // Sort both halves concurrently
     var sortedLeft, sortedRight []int
     go func() {
          sortedLeft = MergeSort(left, &wg2)
     }()

     ____________________________________________________________

     go func() {
          sortedRight = MergeSort(right, &wg2)
     }()

     ____________________________________________________________



     ____________________________________________________________

     // Merge the sorted halves
     return merge(sortedLeft, sortedRight)
}

func main() {
     arr := []int{38, 27, 43, 3, 9, 82, 10}
     var wg sync.WaitGroup
     wg.Add(1)
     sortedArr := MergeSort(arr, &wg)
     wg.Wait()

     fmt.Println("Sorted array:", sortedArr)
}
```